Problem Set 6 by Jimmy Zheng

2023-02-14

1. ISL Chapter 5: Exercise 3. (Your answer should be no more than a couple of sentences for each question).

(a) Explain how k-fold cross-validation is implemented.

We first split the entire traing sample into k disjoint subsets and choose k-1 subsets as training set to fit the model and the subset left out as validation set to estimate test MSE. Repeatedly choosing k-1 subsets k times, we could then use the average MSE to estimate the test MSE of the model.

(b) What are the advantages and disadvantages of k-fold cross validation relative to:

i. The validation set approach? Advantages: k-fold CV has less bias since each training set contains more data points, i.e. (k-1)n/k data points if k>=2. Disadvantages: validation set approach just randomly split the sample into two subsets, which is more computationally convenient.

ii. LOOCV? Advantages: k-fold CV has less variance because the overlap between the training sets in each model is smaller, given the fact that for LOOCV, we are average outputs (MSE) from almost identical (highly correlated) training sets. For k<n, performing k-fold CV is computationlly easier than LOOCV. Disadvantages: k-fold CV has larger bias than LOOCV since each training set contains only (k-1)n/k data points fewer than n-1 in LOOCV.

2. ILS Chapter 5. All parts of Lab Section 5.3

The Validation Set Approach

```
library(ISLR2)
set.seed(1)
train = sample(392, 196)

lm.fit = lm(mpg~horsepower, data = Auto, subset = train)

attach(Auto)
mean((mpg - predict(lm.fit , Auto))[-train ]^2)
```

```
## [1] 23.26601
```

```
lm.fit2 <- lm(mpg ~ poly(horsepower , 2), data = Auto ,subset = train)
mean((mpg - predict(lm.fit2 , Auto))[-train]^2)
```

```
## [1] 18.71646
```

```
lm.fit3 <- lm(mpg ~ poly(horsepower , 3), data = Auto ,subset = train)
mean ((mpg - predict(lm.fit3 , Auto))[-train]^2)
```

```
## [1] 18.79401
```

```
set.seed(2)
train <- sample(392, 196)

lm.fit <- lm(mpg ~ horsepower , subset = train)
mean((mpg - predict(lm.fit , Auto))[-train ]^2)
```

```
## [1] 25.72651
```

```
lm.fit2 <- lm(mpg ~ poly(horsepower , 2), data = Auto , subset = train)
mean((mpg - predict(lm.fit2 , Auto))[-train]^2)
```

```
## [1] 20.43036
```

```
lm.fit3 <- lm(mpg ~ poly(horsepower , 3), data = Auto ,subset = train)
mean((mpg - predict(lm.fit3 , Auto))[-train]^2)
```

```
## [1] 20.38533
```

LOOCV

```
glm.fit <- glm(mpg~horsepower , data = Auto)
coef(glm.fit)
```

```
## (Intercept)  horsepower
##  39.9358610  -0.1578447
```

```
lm.fit <- lm(mpg~horsepower , data = Auto)
coef(lm.fit)
```

```
## (Intercept)  horsepower
##  39.9358610  -0.1578447
```

```
library (boot)
glm.fit <- glm(mpg~horsepower , data = Auto)
cv.err <- cv.glm(Auto , glm.fit)
cv.err$delta
```

```
## [1] 24.23151 24.23114
```

```
cv.error <- rep(0, 10)
for(i in 1:10){
glm.fit <- glm(mpg~poly (horsepower , i), data = Auto)
cv.error[i] <- cv.glm(Auto , glm.fit)$delta[1]
}
cv.error
```

```
##  [1] 24.23151 19.24821 19.33498 19.42443 19.03321 18.97864 18.83305 18.96115
##  [9] 19.06863 19.49093
```

k-fold CV

```
set.seed(17)
cv.error.10 <- rep(0, 10)
for(i in 1:10){
glm.fit <- glm(mpg ~ poly(horsepower , i), data = Auto)
cv.error.10[i] <- cv.glm(Auto , glm.fit , K = 10)$delta[1]
}
cv.error.10
```

```
##  [1] 24.27207 19.26909 19.34805 19.29496 19.03198 18.89781 19.12061 19.14666
##  [9] 18.87013 20.95520
```

Bootstrap

```
# Estimating the Accuracy of alpha
alpha.fn <- function(data , index){
X <- data$X[index]
Y <- data$Y[index]
(var (Y) - cov (X, Y)) / (var (X) + var (Y) - 2 * cov (X, Y))
}

alpha.fn(Portfolio , 1:100)
```

```
## [1] 0.5758321
```

```
set.seed(7)
alpha.fn(Portfolio, sample (100, 100, replace = T))
```

```
## [1] 0.5385326
```

```
set.seed(7)
boot(Portfolio, alpha.fn, R = 1000)
```

```
##
## ORDINARY NONPARAMETRIC BOOTSTRAP
##
##
## Call:
## boot(data = Portfolio, statistic = alpha.fn, R = 1000)
##
##
## Bootstrap Statistics :
##      original      bias    std. error
## t1* 0.5758321 0.001006499  0.08978204
```

```r
#Estimating the Accuracy of a Linear Regression Model
boot.fn <- function(data , index)
coef(lm(mpg ~ horsepower , data = data , subset = index))
boot.fn(Auto , 1:392)
```

```
## (Intercept)  horsepower
##  39.9358610  -0.1578447
```

```r
set.seed(1)
boot.fn(Auto , sample(392, 392, replace = T))
```

```
## (Intercept)  horsepower
##  40.3404517  -0.1634868
```

```r
boot.fn(Auto , sample(392, 392, replace = T))
```

```
## (Intercept)  horsepower
##  40.1186906  -0.1577063
```

```r
set.seed(1)
boot(Auto, boot.fn, 1000)
```

```
##
## ORDINARY NONPARAMETRIC BOOTSTRAP
##
##
## Call:
## boot(data = Auto, statistic = boot.fn, R = 1000)
##
##
## Bootstrap Statistics :
##       original        bias    std. error
## t1* 39.9358610  0.0553942585 0.843931305
## t2* -0.1578447 -0.0006285291 0.007367396
```

```r
summary(lm(mpg ~ horsepower , data = Auto))$coef
```

```
##               Estimate  Std. Error   t value      Pr(>|t|)
## (Intercept) 39.9358610 0.717498656  55.65984 1.220362e-187
## horsepower  -0.1578447 0.006445501 -24.48914  7.031989e-81
```

```r
boot.fn <- function(data , index)
coef(lm(mpg ~ horsepower + I(horsepower^2),data = data , subset = index))


set.seed (1)
boot(Auto , boot.fn, 1000)
```

```
##
## ORDINARY NONPARAMETRIC BOOTSTRAP
##
##
## Call:
## boot(data = Auto, statistic = boot.fn, R = 1000)
##
##
## Bootstrap Statistics :
##         original        bias     std. error
## t1* 56.900099702  3.511640e-02 2.0300222526
## t2* -0.466189630 -7.080834e-04 0.0324241984
## t3*  0.001230536  2.840324e-06 0.0001172164
```

```
summary(lm(mpg ~ horsepower + I(horsepower^2), data = Auto))$coef
```

```
##                     Estimate   Std. Error   t value      Pr(>|t|)
## (Intercept)     56.900099702 1.8004268063  31.60367 1.740911e-109
## horsepower      -0.466189630 0.0311246171 -14.97816  2.289429e-40
## I(horsepower^2)  0.001230536 0.0001220759  10.08009  2.196340e-21
```